Least Squares Sphere Fit

SEPTEMBER 13, 2015

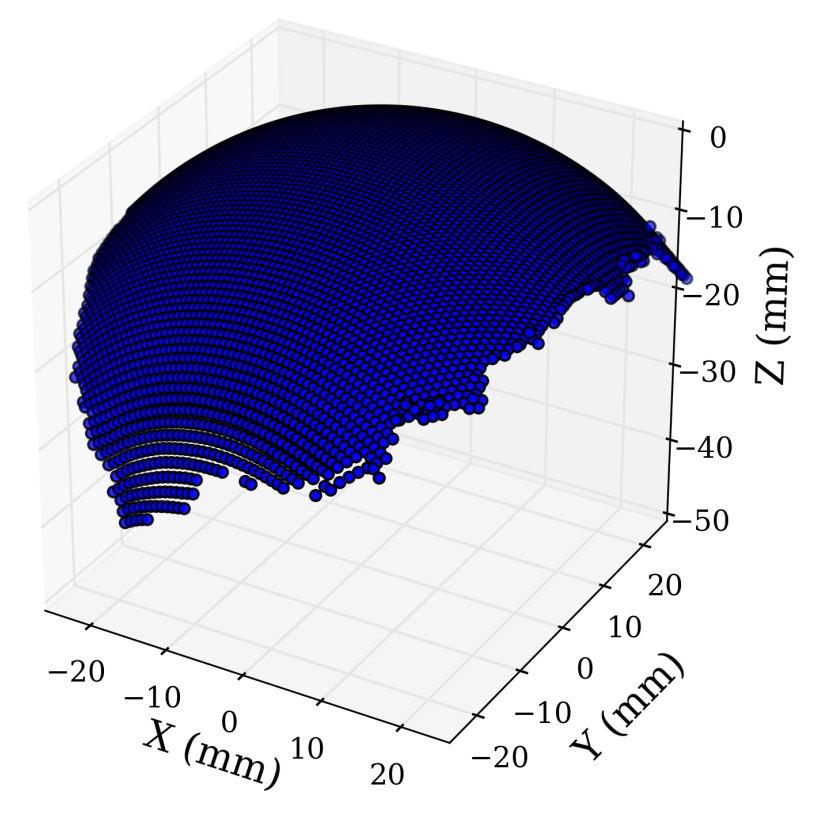
Update: 2020-09-20 If you are interested in fitting ellipsoids or formulating other least squares problems check out this new post.

Update: 2018-04-22 I've uploaded the data and a demo Python file here.

Update: 2016-01-22 I have added the code I used to make the plot of the 3D data and sphere!

It may not be intuitive to fit a sphere to three dimensional data points using the least squares method. This post demonstrates how the equation of a sphere can be rearranged to formulate the least squares problem. A Python function, which determines the sphere of best fit, is then presented.

So let's say you have a three dimensional data set. The data points plotted in three dimensional space resemble a sphere, so you'd like to know the sphere that would fit your data set the best. Well I have a sample data set that is well suited for a spherical fit using the least squares method. A plot of data points in three dimensional space can be seen in the following image. Now let's go through the process fitting a sphere to this data set.



The general equation of a sphere in $x$, $y$, and $z$ coordinates can be seen below. The center point of the sphere with radius r is found at the point (x_0, y_0, z_0). We must rearrange the terms of the equation in order to use the least squares method.

$$(x - x_0)^2 + (y - y_0)^2 + (z - z_0)^2 = r^2$$

After expanding and rearranging the terms, the new equation of a sphere is expressed below. This equation can now be expressed in vector/matrix notation.

$$x^2 + y^2 + z^2 = 2xx_0 + 2yy_0 + 2zz_0 + r^2 - x_0^2 - y_0^2 - z_0^2$$

The $\vec{f}$ vector, the A matrix, and the $\vec{c}$ vector represents the consolidated terms of the expanded sphere equation. The terms x_i, y_i, and z_i represent the first data point, while x_n, y_n, and z_n represent the last data point in the data set.

$$\vec{f} = \begin{bmatrix} x_i^2 + y_i^2 + z_i^2 \\ x_{i+1}^2 + y_{i+1}^2 + z_{i+1}^2 \\ \vdots \\ x_n^2 + y_n^2 + z_n^2 \end{bmatrix}$$

$$A = \begin{bmatrix} 2x_i & 2y_i & 2z_i & 1 \\ 2x_{i+1} & 2y_{i+1} & 2z_{i+1} & 1 \\ \vdots & \vdots & \vdots & \vdots \\ 2x_n & 2y_n & 2z_n & 1 \end{bmatrix}$$

$$\vec{c} = \begin{bmatrix} x_0 \\ y_0 \\ z_0 \\ r^2 - x_0^2 - y_0^2 - z_0^2 \end{bmatrix}$$

We now have an over-determined system suitable for the least squares method of a spherical fit. The new equation is seen below. The fit determines the best $\vec{c}$ from the data points. We can then calculate the sphere's radius using the terms in the $\vec{c}$.

$$\vec{f} = A\vec{c}$$

We can use the above equation to define a simple Python function that will fit a sphere to $x$, $y$, and $z$ data points. The Python NumPy library includes a least squares function that is used to determine the best $\vec{c}$. The function then returns the radius and center coordinates of the sphere.

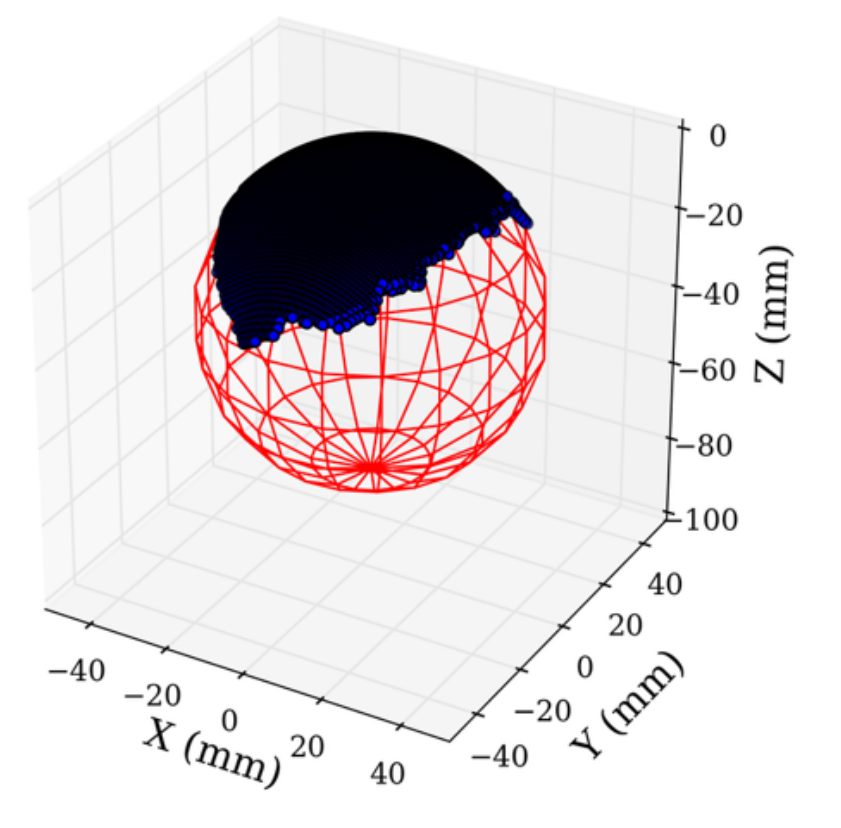
```python
import numpy as np
#       fit a sphere to X,Y, and Z data points
#       returns the radius and center points of
#       the best fit sphere
def sphereFit(spX,spY,spZ):
    #   Assemble the A matrix
    spX = np.array(spX)
    spY = np.array(spY)
    spZ = np.array(spZ)
    A = np.zeros((len(spX),4))
    A[:,0] = spX*2
    A[:,1] = spY*2
    A[:,2] = spZ*2
    A[:,3] = 1

    #   Assemble the f matrix
    f = np.zeros((len(spX),1))
    f[:,0] = (spX*spX) + (spY*spY) + (spZ*spZ)
    C, residules, rank, singval = np.linalg.lstsq(A,f)

    #   solve for the radius
    t = (C[0]*C[0])+(C[1]*C[1])+(C[2]*C[2])+C[3]
    radius = math.sqrt(t)

    return radius, C[0], C[1], C[2]
```

We can easily fit a sphere to our original data set using this function. The resulting sphere of best fit plotted with the original data points can be seen in the following image.



The above 3D plot of the fitted sphere and data was created using the following code.

```python
from matplotlib import rcParams
rcParams['font.family'] = 'serif'
#   3D plot of the
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

r, x0, y0, z0 = sphereFit(correctX,correctY,correctZ)
u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]
x=np.cos(u)*np.sin(v)*r
y=np.sin(u)*np.sin(v)*r
z=np.cos(v)*r
x = x + x0
y = y + y0
z = z + z0

#   3D plot of Sphere
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(correctX, correctY, correctZ, zdir='z', s=20,
c='b',rasterized=True)
ax.plot_wireframe(x, y, z, color="r")
ax.set_aspect('equal')
ax.set_xlim3d(-35, 35)
ax.set_ylim3d(-35,35)
ax.set_zlim3d(-70,0)
ax.set_xlabel('$x$ (mm)',fontsize=16)
ax.set_ylabel('\n$y$ (mm)',fontsize=16)
zlabel = ax.set_zlabel('\n$z$ (mm)',fontsize=16)
plt.show()
plt.savefig('steelBallFitted.pdf', format='pdf', dpi=300,
bbox_extra_artists=[zlabel], bbox_inches='tight')
```

Please let me know if you found this post useful!

Please cite this work as:

```
@book{Jekel2016,
 author = {Jekel, Charles F},
 booktitle = {Obtaining non-linear orthotropic material
             models for pvc-coated polyester via inverse
             bubble inflation},
 chapter = {Appendix A},
 organization = {Stellenbosch University},
 pages = {83--87},
 title = {Digital Image Correlation on Steel Ball},
 url = {https://hdl.handle.net/10019.1/98627},
 year = {2016}
}
```

Charles Jekel
Engineer. Regression. Optimization.
cj@jekel.me
Home CV

Comments

ALSO ON JEKEL.ME

Detect faces using facenet in Python
9 years ago · 12 comments
The TensorFlow implementation of facenet is an extremely powerful …

USA GDP and GNI per capita adjusted for …
10 years ago · 1 comment
Plots were created of the USA's GDP per capita and GNI per capita from 1960 …

Fitting a pi linear func
9 years ago ·
A Python libr continuous p functions to

25 Comments 1 Login ▾

Join the discussion…

LOG IN WITH

OR SIGN UP WITH DISQUS ?

Name

♡ 6 • Share Best Newest Oldest

Adnan Munawar
7 years ago
Thank you for the clean and minimalist implementation
1 0 Reply

Steve Boege
8 years ago
Tested the function by adding the following to the demo.py right after line 36. Results matched expectations. Thanks for making this public!
X,Y = np.meshgrid(np.arange(-16, 9, 6), np.arange(-34, 27, 6))
XX = X.flatten()
YY = Y.flatten()
myROC = 100
myX0 = -4
myY0 = 2
myZ0 = myROC
Z = myZ0 - np.sqrt(myROC**2 - (X - myX0)**2 - (Y-myY0)**2)
correctX = XX
correctY = YY
correctZ = Z.flatten()
1 0 Reply

Jonathan Zender
2 years ago
is there a possibilty to do the fit with a fixed radius?
0 0 Reply

Max Herrmann
3 years ago
This is just ingenious.
0 0 Reply

> **Charles Jekel** Mod → Max Herrmann
> 3 years ago
> Thanks!
> 0 0 Reply

Young Wang
3 years ago
Great post, I wonder if you can have a fixed origin and estimate the radius?
0 0 Reply

> **Charles Jekel** Mod → Young Wang
> 3 years ago
> Yes it's not too difficult. Move all terms into f basically. A becomes a n_data x 1 matrix, and c is just r^2. Give it a try and let me know.
> 0 0 Reply

Yogun
5 years ago
do you have the same but for ellipsoid?
0 0 Reply

> **Charles Jekel** Mod → Yogun
> 5 years ago
> Hi Yogun,
>
> Thanks for the suggestion. I ended up creating a new blog post on fitting ellipsoids. You should check it out.
>
> https://jekel.me/2020/Least...
>
> Thanks,
> CJ
> 0 0 Reply
>
> > **Faisal Ali Khan** → Charles Jekel
> > 4 years ago
> > Could you do one for a Poly-superquadratic surface.
> > 0 0 Reply

> **Charles Jekel** Mod → Yogun
> 5 years ago
> I don't have an implementation now, but it should be pretty straight forward to implement (actually a bit easier than the sphere fit). If you get lost, let me know. Perhaps I could do a quick post on how to fit general ellipsoids.
> 0 0 Reply

Daavid
5 years ago
Hi. Thank you for this. I'm a python novice but I'd like to use this and I'm a bit lost getting it to work. I have my x, y, z files in .txt format and would appreciate a guide in how to make it work. Thank you
0 0 Reply

> **Charles Jekel** Mod → Daavid
> 5 years ago
> I normally use np.loadtxt to get read plain text files. Check out the documentation here https://numpy.org/doc/stabl...
> 0 0 Reply

Adrien Boyer
7 years ago
Great function ! I have tested it on 2D profile and is yet the most accurate and also resilient to noise so far.
Thanks a lot !
0 0 Reply

Steve Boege
8 years ago edited
Very helpful, thanks. Would you be willing to post the test data so we can step through the code and see what happens at each step? In particular, does the data get passed in as 2D arrays? Or as 1D, flattened 2D arrays?

Also, for people not familiar with numpy.mgrid, I found the following, "...if the step length is a complex number (e.g. 5j), then the integer part of its magnitude is interpreted as specifying the number of points to create between the start and stop values, where the stop value is inclusive." https://docs.scipy.org/doc/...
0 0 Reply

> **Charles Jekel** Mod → Steve Boege
> 8 years ago
> Done. Check out demo.py and the .npy files here https://github.com/cjekel/c...
>
> spX, spY, spZ should be 1D arrays. If you have 2D array data, then flattening should be fine.
> 0 0 Reply

prasun kumar
8 years ago
Hi Charles...

Really nice and informative post. You have saved alot of time for me. Thank you very much....

I was curious to know about the RMSD or mean deviation from the fit for a given a dataset.

Thank you in advance
0 0 Reply

> **Charles Jekel** Mod → prasun kumar
> 8 years ago
> You can't solve for those formulations in a least squares fit. I'd set up a global optimizer to minimize x0, y0, z0 and r such that you get the smallest RMSD or mean deviation.
> 0 0 Reply

nuhash
9 years ago
Hi, I just wanted to check something. In your example you used data without any noise. If the data points didn't perfectly fit a sphere would this technique still be applicable?
0 0 Reply

> **Charles Jekel** Mod → nuhash
> 9 years ago
> There is noise in the data I used, it's just relatively small. The data comes from real measurements on a ball bearing.
>
> Using this method for noisy data is perfectly fine. Mathematically the method gives you the best possible (lowest error) sphere that could possibly fit to your data, regardless of whatever true function your data is from.
> 0 0 Reply

Patrick Na
9 years ago
Hey, thanks for this great post. After receiving the radius and the three center coordinates, how did you plot the segment of the sphere fitted? Thanks in advance!
0 0 Reply

> **Charles Jekel** Mod → Patrick Na
> 9 years ago
> I just added the code I used to make the plot. If you can't follow it just let me know.
> 0 0 Reply

Marco Grubert
9 years ago
Very nice!
0 0 Reply

Paul
10 years ago
Thanks for the simple explanation and code. It works perfectly!

I'd like to have the option of fixing the radius to a known value, and estimate (x0, y0, z0), but I can't get it to work. I tried moving the r^2 term to the left-hand side (subtracting it from f). However, this doesn't change the resulting x0, y0 and z0 (the first three terms of c), and only changes the fourth term.

Do you have any ideas? How should I arrange the equations to do this?
0 0 Reply

> **Charles Jekel** Mod → Paul
> 10 years ago
> Hi Paul,
>
> I don't think you can use the least squares method to fit a sphere with a fixed radius. Take a look at the c vector. If you remove r^2 from the fourth term, you'll find that the fourth term may not be evaluated with the determined x0, y0, z0 center points. I can't think of a way to avoid this at the moment.
>
> My gut says this would require an optimization run, where you try to minimize some error. The optimization would result in the x0, y0, and z0 that returns the minimum error. The question now lies in what error would you be trying to minimize? The minimum distance between the sphere surface and each data point? You'd have to look at derivatives just to determine the minimum distance from a data point to the sphere's surface. Though this wouldn't be impossible, it gets really complicated quickly.
>
> Take a look at this paper which describes a fixed point iteration method http://www.sci.utah.edu/~ba...
>
> Thanks,
> CJ
> 0 0 Reply